Drop, Reflect and Reuse a Quantum Temporary

ANONYMOUS AUTHOR(S)

A quantum program often allocates a temporary register, uses it, and drops it. Compilers usually reclaim its qubits by uncomputing it, running backwards the gates that acted on it. This can repeat much of the work even when the drop was already safe. We specify a safe drop by what can be observed: just before the drop, no context can tell the program from one that prepared the temporary without the input. This holds exactly when the temporary ends in one fixed pure state at every input, and then at most one reflection on it alone restores its starting state. The test extends to programs that measure, one record of outcomes at a time, and to borrowed registers. A prototype compiler to OpenQASM 3 computes the gate and reuses the freed wire.

1 Specifying a safe drop

Qubits are too scarce to waste, so a quantum program must reclaim its temporaries. The standard technique, uncomputation, runs the computation that produced a temporary backwards, which can take as many gates as the computation itself [12, §1]. Whether a temporary may be safely discarded is a separate question, which languages and tools such as Silq and Unqomp settle with conservative rules for automatic reclamation that can reject temporaries that are in fact safe [2, §4.4][15, §6.2]. We instead define safety semantically, within the language, by comparing the program just before the drop with one that prepared the temporary independently of the input. When no context can distinguish these programs, the drop is safe. For a freshly allocated temporary without measurement, this means it ends in one fixed pure state at every input, and knowing that state lets us restore its allocation state with at most one reflection on the temporary alone. The backward run is then unnecessary, however long the computation that produced the temporary. With measurement the comparison applies to one record of outcomes at a time, keeping its weight.

Let p allocate `t` in a pure state ψ, run a measurement-free computation M on `t` and the input `v`, and drop `t`. An independent witness $\mathcal{S}(P,U)$ uses P to prepare `t` without `v`, while U acts on `v` and fresh registers for M's other temporaries:

$$p(\texttt{v}) = \texttt{new t : [d] @ ψ; M; drop t; return v},$$

$$\mathcal{S}(P,U)(\texttt{v}) = \texttt{new t : [d] @ ψ; P t; U v; drop t; return v}.$$

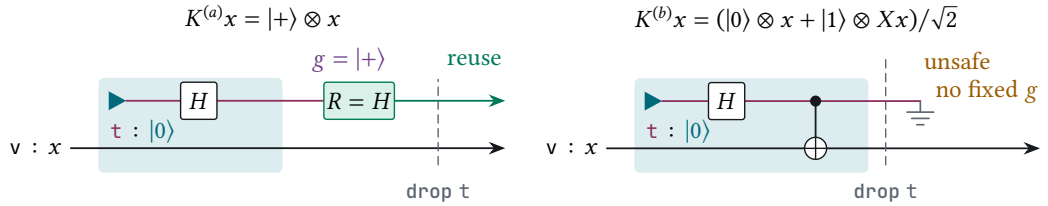
The drop is *safe* if $p \equiv \mathcal{S}(P,U)$ for one witness on all inputs, meaning that no context can distinguish the programs stopped before their drops, with `t` and all other registers returned to the context.

Shading in the circuits marks the *exposed prefix*, which includes allocation and stops just before a source `drop` *site*. Its linear map K sends the input state x of `v` to the joint state there, with the dropped *block* first. A return gate R, inserted just before a safe `drop t`, restores `t`'s allocation state so its wire can serve a later `new`.

Two programs on a qubit register `v` show the difference:

```
prog a (v : [2]) : [2] {
  new t : [2] @ |0>;
  H t;
  drop t; return v }
```

```
prog b (v : [2]) : [2] {
  new t : [2] @ |0>;
  H t; CX t v;
  drop t; return v }
```



Here `new t : [2] @ |0>` allocates a qubit `t`, of dimension 2, in $|0\rangle$, and `drop t` ends its scope without changing the joint state. In program (a), the Hadamard gate H puts `t` in the plus state $|+\rangle$, an equal superposition of $|0\rangle$ and $|1\rangle$, whatever `v` held, and another H would restore $|0\rangle$ without touching `v`. Program (b) adds `CX t v`, which flips `v` when `t` is 1. With `v` initially in $|0\rangle$, program (b) leaves `t` and `v` in a Bell state, an equal superposition of $|00\rangle$ and $|11\rangle$ that no two separate operations can prepare, so dropping `t` leaves `v` mixed. In both programs, `t`'s lifetime ends at the drop, so scope checks accept both. Silq and Unqomp's `qfree` rules require producing gates to map basis states to basis states, so neither automatically uncomputes program (a), where H creates a superposition from $|0\rangle$ [2, §4.4][15, §6.2].

2 Checking the equivalence

$$K^{(a)} = \underbrace{\frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix}}_{H \otimes I} \cdot \underbrace{\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}}_{|0\rangle \otimes I} = \underbrace{\frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}}_{K^{(a)}} \begin{matrix} K_0 \\ K_1 \end{matrix}$$

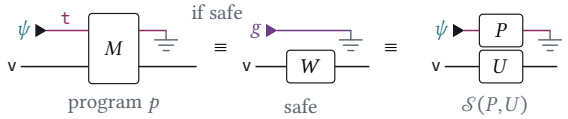
slices $I/\sqrt{2}, I/\sqrt{2}$

$$\underbrace{\qquad\qquad}_{K^{(b)} = \mathrm{CX}\,K^{(a)}} \quad \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{matrix} K_0 \\ K_1 \end{matrix}$$

slices $I/\sqrt{2}, X/\sqrt{2}$

The matrix $|0\rangle \otimes I$ acts first, on the input x. The blue rows give the part of the joint state labeled $|0\rangle$ on `t`, and the gold rows give the part labeled $|1\rangle$. We call their maps the *slices* K_0 and K_1. The joint state contains both parts, so neither slice is selected.

A *context* is a closed program with a hole for the prefix. It supplies the input, possibly entangled with a reference, may apply gates to the returned registers, and reads every register, recording each outcome's weight. For example,




```
new v : [2] @ |0>; (t,v) = { □ }; CX t v; H t; return (t,v).
```

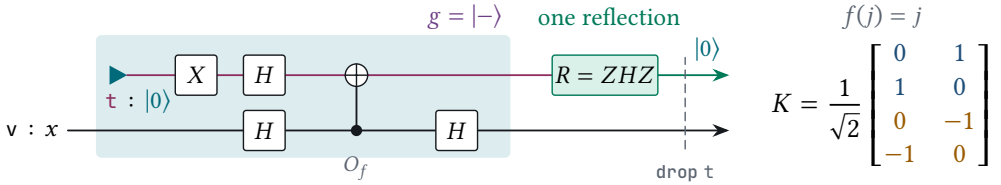
The context reverses program (b)'s gates and reads 00 with weight 1. With an independent witness in the hole, the same context reads 00 at most half the time because the witness leaves `t` and `v` uncorrelated. To match a witness before the drop, p must also leave `t` in one

fixed pure state g, independently of the input. If g_i is the coefficient of $|i\rangle$ in g, the slice K_i must therefore be $g_i W$ for the same map W on the rest. Thus $Kx = g \otimes Wx$ at every input x. In program (a) both slices are equal, so g is $|+\rangle$ and W is the identity. In program (b) the slice at 1 is the slice at 0 followed by X, and no number turns the identity into X. Triangles mark allocation, and the ground symbol marks where the source ends a register's scope.

THEOREM 2.1 (WITHOUT MEASUREMENT). *For a well-typed, measurement-free program p, let K be its exposed map at a drop of only allocated temporaries and ψ their pure allocation state. The following are equivalent.*

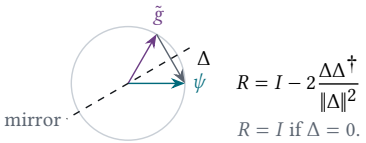
(1) $p \equiv \mathcal{S}(P, U)$ for some independent witness.
(2) $K = g \otimes W$ for a unit vector g on the block and an isometry W to all other registers.
(3) One unitary on the block alone restores ψ at every input.

Deutsch–Jozsa shows why slices need only be proportional, not equal. Its oracle O_f flips the temporary when $f(z) = 1$. Prepared in the minus state $|-\rangle$, the temporary is only negated by a flip, so the oracle leaves it in $|-\rangle$ and puts the sign $(-1)^{f(z)}$ on the data state $|z\rangle$. These signs define a data gate D_f, with $D_f|z\rangle = (-1)^{f(z)}|z\rangle$. The two Hadamards on the data in the pictured one-qubit program make its full action $W = HD_fH$. Since $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$, the upper slice is $W/\sqrt{2}$ and the lower is $-W/\sqrt{2}$. The slices are proportional for every f, and `t` ends in $|-\rangle$ at every input.



Reversing the preparation applies H and then X to `t` alone. Knowing $g = |-\rangle$ also lets us restore `t` with the single reflection $R = ZHZ$, so neither method touches the data.

To reuse `t`, we build the return gate R from g, our *certificate*. The factorization fixes g only up to a phase, so choose $\tilde{g}$ whose overlap with ψ is real and nonnegative, moving the phase into W. The difference $\Delta = \psi - \tilde{g}$ points from the certificate to the allocation state. If $\tilde{g}$ is ψ, no gate is needed, as safe use [12, Eq. (1)] and clean safety [11, Def. 1] require of every clean ancilla. Otherwise the Householder reflection R through the pictured mirror orthogonal to Δ [9, p. 340, Eqs. (3)–(4)] sends $\tilde{g}$ to ψ on the temporary alone, leaving the rest unchanged. The witness's preparation gate P maps ψ to $\tilde{g}$. This reflection is its own inverse, so we can choose $P = R$. For any witness, undoing P gives a return gate $R = P^\dagger$, since a context must then find ψ in p as it does in the witness. For program (a), R is H, and Grover's diffusion [7, §4] is $-R$ when Δ is uniform.



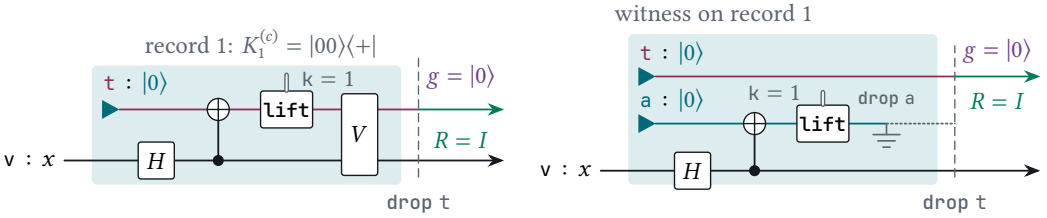
3 One measurement record at a time

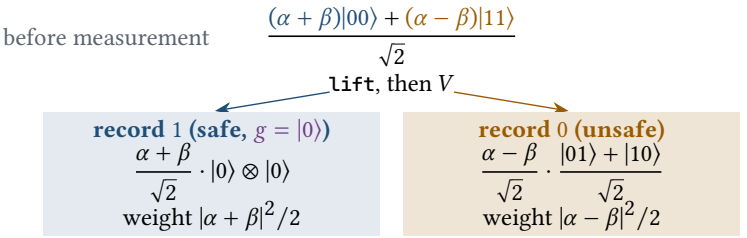
With measurement, a program can be safe on one sequence of outcomes and unsafe on another. Our `lift` measures a register and, as in dynamic lifting [6], binds an outcome that

only classical `if` guards can read. In `lift[{0}] t`, outcome 1 means that `t` was read as 0, and outcome 0 means it was read as 1. A *record* o lists the outcomes along one path through the program. Its exposed map K_o is the denotation along that path, left unnormalized, so that $\|K_o x\|^2$ is the probability of the path at a normalized input x.

```
prog c (v : [2]) : [2] {
  new t : [2] @ |0>;
  H v; CX v t;
  k = lift[{0}] t; V t v;
  drop t; return v }
```

```
prog witness (v : [2]) : [2] {
  new t : [2] @ |0>;
  new a : [2] @ |0>; H v; CX v a;
  k = lift[{0}] a;
  drop a; drop t; return v }
```





For input $x = \alpha|0\rangle + \beta|1\rangle$, applying H to `v` and copying into `t` gives the joint state shown above the two records, with `t` first. The `lift` keeps one of the two terms, so the squared magnitude of that term's coefficient is the weight of its record. Gate V then leaves $|00\rangle$ unchanged on record 1 and turns $|11\rangle$ into a Bell state on record 0.

On record 1, program (c)'s exposed prefix $K_1^{(c)} = |00\rangle\langle+|$ has proportional slices $W_1 = |0\rangle\langle+|$ and 0, so `t` already holds $|0\rangle$ and $R = I$. The map W_1 keeps only the $|+\rangle$ component of the input, so it never lengthens a vector and is a *contraction*. For example, W_1 gives $|0\rangle$ with weight 1 at $x = |+\rangle$, but $|0\rangle/\sqrt{2}$ with weight $1/2$ at $x = |0\rangle$. Normalizing would give $|0\rangle$ in both cases, hiding the dependence of record 1's weight on the input.

The witness's allocations of `t` and `a` in $|0\rangle$ and its gates `H v; CX v a` preserve total weight, so matching record 1's weight also requires a measurement. Measuring the independent `t` would give the same probability at every input, so the witness measures `a` instead. Outcome 1 of `lift[{0}] a` leaves `t` and `a` in $|0\rangle$ and `v` in $W_1 x$, matching record 1 with its weight. Program (c)'s unsafe record 0 still prevents reuse.

The same construction works whenever $K_o x = g \otimes W_o x$ with W_o a contraction. The helper's other outcome carries the squared length missing from $W_o x$. With fresh registers to match the input and output spaces, the two outcomes preserve norm and extend to a unitary U [18, Thm. 2]. One `lift` selects $W_o x$, while P prepares g from ψ on the block.

We call record o *safe* and write $p \equiv_o \mathcal{S}(P,U)$ when no context distinguishes these selected, unnormalized maps before the drop, ignoring the fixed helper, whatever the other records do.

Theorem 3.1 (on a selected record). *Let K_o be the exposed map on a record o reaching a drop of only allocated temporaries in a well-typed program p. The following are equivalent.*
(1) $p \equiv_o \mathcal{S}(P,U)$ for some independent witness using one `lift`.
(2) $K_o = g \otimes W_o$ for a unit vector g on the block and a contraction W_o to all other registers.
(3) One unitary on the block alone restores its pure allocation state ψ at every input on o.
Zero maps satisfy the three conditions, and return gates may differ between records.

After `drop t`, a context that ignores the record sees the *summed channel*, the mixture on returned registers and a reference weighted by record probabilities [21, §2.3.2]. For example, allocate `t` in $|+\rangle$ and compare `CZ t v; k = lift[{0}] t` with `k = lift[{0}] t; CX v t`, both followed by `drop t; return v`. Here `CZ t v` negates $|11\rangle$ and leaves the other basis states unchanged. Measuring `t` last leaves it in $|0\rangle$ on record 1 and $|1\rangle$ on record 0, so both drops are safe. When measurement comes first, the subsequent `CX v t` leaves a Bell state on either record at input $x = |+\rangle$, so both drops are unsafe. Yet both programs induce the same standard-basis measurement on `v`, and each record has weight $1/2$ at every input.

4 Blocks and loans

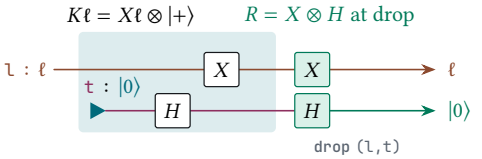
Dropping a tuple tests the named registers together. A Bell pair prepared in two fresh temporaries and left untouched is safe as a block of two, since the pair has one fixed state vector, though each half alone has the mixed state $I/2$ and so supplies no pure certificate.

We also consider blocks containing *loans*, registers borrowed in unknown states, possibly entangled with their caller, who expects them back unchanged. The witness $\mathcal{S}(P,U)$ therefore receives the loan's state ℓ, allocates the temporaries beside it (the map $J\ell = \ell \otimes \psi$, with loans first), applies P to the whole block, and lets U act only outside it. The fixed certificate g becomes an isometry B from the loan's space into the block's. The drop is then safe exactly when $K = B \otimes W$. The witness exists because a unitary P of the block carries the isometry J onto B, $PJ = B$, and the return gate $R = P^\dagger$ satisfies $RB = J$.

With measurement we compare using $B_o \otimes W_o$ on each record, and B_o and the return gate may depend on the record. If the program measures nothing, or the caller's loan and reference together are uncorrelated with its other inputs, the gate restores their normalized joint state on each record that occurs. Otherwise a record can change it.

For program `lend`, the exposed prefix is $K\ell = B\ell = X\ell \otimes |+\rangle$, so the inserted $R = X \otimes H$ returns `l` in its unknown state ℓ and restores `t` to $|0\rangle$.

```
prog lend (loan l : [2]) : 1 {
  new t : [2] @ |0>;
  H t; X l;
  drop (l,t); return () }
```



5 Scope and further work

The language is linear and first order. We prove progress and preservation for its abstract machine with lifting and classical guards, and extend it to restore a safe block on each record and pool its temporaries for a later `new` of matching dimension and state. Reuse preserves the records, their weights and, on each record, the reduced state of the returned registers. Our Haskell checkers compute exactly. One builds K_o at one-temporary sites. The other tests basis inputs and their sum for a fixed temporary state, or checks a loan block for an isometric B, on each record. Both are exponential in the number of qubits, but even with only basis allocations and X, `CX` and Toffoli gates, deciding whether a drop of temporaries is safe on a record is coNP-complete (compare [12, Cor. 3.1]). Such a drop is unsafe exactly when two surviving basis inputs leave different block values.

Our OpenQASM 3 compiler uses the basis-input and loan-block check, computes return gates, and reuses basis-allocated blocks safe on every record. If program (a) ends with `new y : [2] @ |1>; return (v,y)` instead, its circuit uses two qubits where fresh allocation uses three: the return gate H restores `t`, and X prepares `y` on `t`'s wire. Measuring `t` and resetting it to $|0\rangle$, as qubit-reuse compilers do [4, 10], would save the qubit too, but a reset destroys a loan's state, which the return gate hands back. Only basis allocations compile, program (c)'s V is unsupported, and no loan may be dropped after a `lift` or an `if`. Return gates are numerical, and for a general loan return, $RB = J$ is checked entrywise, to within 10^{-9}.

Testing the state a temporary ends in, not the gates that produced it, our criterion accepts the balanced Deutsch–Jozsa drop that Silq and Unqomp refuse and Qutes does not reclaim (neither dead nor output-isolable [5, Defs. 2–6]). Qurts puts lifetimes in types [8, §3] and Reqomp trades qubits for gates [16, §4], and both reclaim by uncomputing. ReQWIRE verifies that an ancilla is restored [17, §4], as Q# [14] and fractional types [3] require. Local rotations repair separable errors on a dirty ancilla [11, §5.2, Alg. 2], where our one gate per record restores a whole block. On measurement-free circuits with $|0\rangle$ ancillas, the validity test of slices summing to a unitary [12, Def. 3.1] is incomparable with safety, as program (a)'s slices sum to $\sqrt{2}I$, yet its drop is safe, and those of `new t : [2] @ |0>; CX v t; drop t` sum to I, yet its drop is unsafe. QBorrow needs the loan back untouched after each denoted operation [19, Def. 5.1] and Bona after each borrowing body [22, Def. 3.2], and separation logics prove its return by hand [13, 20]. Our return gate undoes any isometry B.

We did not consider loops or quantum recursion. What certifies a temporary dropped after a loop, possibly in superposition, is open. We also took every unitary to be a gate, but a fixed gate set may lack the return gate. Circuits of X, `CX`, Toffoli and $H \otimes H$ have dyadic entries [1, Cor. 5.6]. After $H \otimes H$ on `t` and `v` the drop of `t` is safe, yet no such circuit on `t` alone restores it, since that needs the non-dyadic entries $\pm 1/\sqrt{2}$.

References

[1] Matthew Amy, Andrew N. Glaudell, and Neil J. Ross. 2020. Number-Theoretic Characterizations of Some Restricted Clifford+T Circuits. *Quantum* 4, Article 252 (2020). doi:10.22331/q-2020-04-06-252

[2] Benjamin Bichsel, Maximilian Baader, Timon Gehr, and Martin Vechev. 2020. Silq: A High-Level Quantum Language with Safe Uncomputation and Intuitive Semantics. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '20)*. ACM, 286–300. doi:10.1145/3385412.3386007

[3] Chao-Hong Chen, Vikraman Choudhury, Jacques Carette, and Amr Sabry. 2020. Fractional Types: Expressive and Safe Space Management for Ancilla Bits. In *Reversible Computation — 12th International Conference, RC 2020 (Lecture Notes in Computer Science, Vol. 12227)*. Springer, 169–186. doi:10.1007/978-3-030-52482-1_10

[4] Matthew DeCross, Eli Chertkov, Megan Kohagen, and Michael Foss-Feig. 2023. Qubit-Reuse Compilation with Mid-Circuit Measurement and Reset. *Physical Review X* 13, 4 (2023), 041057. doi:10.1103/PhysRevX.13.041057

[5] Simone Faro, Francesco Pio Marino, and Gabriele Messina. 2026. Reversible Lifetime Semantics for Quantum Programs. arXiv:2603.14538 [quant-ph] (numbered as in version 1).

[6] Peng Fu, Kohei Kishida, Neil J. Ross, and Peter Selinger. 2023. Proto-Quipper with Dynamic Lifting. *Proceedings of the ACM on Programming Languages* 7, POPL, Article 11 (2023), 26 pages. doi:10.1145/3571204

[7] Lov K. Grover. 1996. A Fast Quantum Mechanical Algorithm for Database Search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing (STOC)*. ACM, 212–219. doi:10.1145/237814.237866

[8] Kengo Hirata and Chris Heunen. 2025. Qurts: Automatic Quantum Uncomputation by Affine Types with Lifetime. *Proceedings of the ACM on Programming Languages* 9, POPL, Article 6 (2025), 28 pages. doi:10.1145/3704842

[9] Alston S. Householder. 1958. Unitary Triangularization of a Nonsymmetric Matrix. *J. ACM* 5, 4 (Oct. 1958), 339–342. doi:10.1145/320941.320947

[10] Fei Hua, Yuwei Jin, Yanhao Chen, Suhas Vittal, Kevin Krsulich, Lev S. Bishop, John Lapeyre, Ali Javadi-Abhari, and Eddy Z. Zhang. 2023. CaQR: A Compiler-Assisted Approach for Qubit Reuse through Dynamic Circuit. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS '23)*. 59–71. doi:10.1145/3582016.3582030

[11] Jiqi Li, Jingyi Mei, Wang Fang, and Ji Guan. 2026. Formal Verification of Quantum Ancilla Safety. In *Computer Aided Verification*. Springer Nature Switzerland, 326–348. doi:10.1007/978-3-032-32537-2_16 (numbered as in arXiv:2608.13099v1).

[12] Chenke Liu, Li Zhou, and Boning Meng. 2026. Quantum Uncomputation of Clean and Dirty Ancilla Qubits. *Proceedings of the ACM on Programming Languages* 10, OOPSLA2 (2026), 2270–2298. doi:10.1145/3839528 (numbered as in arXiv:2608.09578v2).

[13] Yusuke Matsushita, Kengo Hirata, Ryo Wakizaka, and Emanuele D'Osualdo. 2026. RapunSL: Untangling Quantum Computing with Separation, Linear Combination and Mixing. *Proceedings of the ACM on Programming Languages* 10, POPL, Article 6 (2026). doi:10.1145/3776648

[14] Microsoft. 2025. Quantum Memory Management in Q#. Microsoft Learn, Azure Quantum documentation. https://learn.microsoft.com/en-us/azure/quantum/user-guide/language/statements/quantummemorymanagement

[15] Anouk Paradis, Benjamin Bichsel, Samuel Steffen, and Martin Vechev. 2021. Unqomp: Synthesizing Uncomputation in Quantum Circuits. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '21)*. ACM, 222–236. doi:10.1145/3453483.3454040

[16] Anouk Paradis, Benjamin Bichsel, and Martin Vechev. 2024. Reqomp: Space-Constrained Uncomputation for Quantum Circuits. *Quantum* 8 (2024), 1258. doi:10.22331/q-2024-02-19-1258

[17] Robert Rand, Jennifer Paykin, Dong-Ho Lee, and Steve Zdancewic. 2019. ReQWIRE: Reasoning about Reversible Quantum Circuits. *Electronic Proceedings in Theoretical Computer Science* 287 (2019), 299–312. doi:10.4204/EPTCS.287.17

[18] P. L. Robinson. 2018. Julia Operators and Halmos Dilations. arXiv:1803.09329 doi:10.48550/arXiv.1803.09329

[19] Bonan Su, Li Zhou, Yuan Feng, and Mingsheng Ying. 2026. Borrowing Dirty Qubits in Quantum Programs. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26)*. 274–289. doi:10.1145/3779212.3790134 (numbered as in arXiv:2508.17190v1).

[20] Bonan Su, Li Zhou, Yuan Feng, and Mingsheng Ying. 2026. An Interpretation of Bunched Logic for Reasoning about Heap-Manipulating Quantum Programs. arXiv:2409.10153 [cs.PL] v2, 5 Aug 2026; v1 (16 Sep 2024) was titled "BI-based Reasoning about Quantum Programs with Heap Manipulations".

[21] John Watrous. 2018. *The Theory of Quantum Information.* Cambridge University Press. doi:10.1017/9781316848142 (numbered as in the 2018 prepublication draft).

[22] Xiaoquan Xu, Chenke Liu, Boning Meng, Zihao Shen, and Li Zhou. 2026. Bona: Automatic Management of Dirty Ancilla Borrowing in Quantum Circuits. arXiv:2608.08765 [cs.PL] (numbered as in version 1).